



AgentTollgate

The 60-second paywall for the agentic economy.

Drop CheckoutWithLocus on any HTTP API. AI agents preflight, pay in USDC, and consume — with policy enforcement, reputation-aware pricing, and live revenue analytics.

402-style flow

USDC on Base

MCP-native

Policy-as-code

Locus · Paygentic Hackathon · Week 3 — CheckoutWithLocus track

Open-source · MIT

Stripe doesn't fit the next buyer.

10×

agent traffic vs. human traffic on top APIs by 2027
(a16z)

0

card-based checkouts an LLM agent can complete
unattended

\$0.001

the price point Stripe simply can't profitably settle

AI agents are about to call APIs the way humans called websites — programmatically and constantly.

But the buyer isn't a card-holding human in a browser. It's a script with a USDC wallet, a budget, and no patience for redirects.

- **Stripe Checkout:** designed for humans + cards. No spend caps. No reputation. No 402.
- **Crossmint / Coinbase Commerce:** USDC settlement, but web-first. Fees and UX kill sub-cent calls.
- **DIY rolling-your-own:** every merchant rebuilds caps, rate limits, reputation, MCP. Wasted weeks.

A drop-in toll for any HTTP endpoint.

1

Paste an upstream URL + a price

Anywhere from 0.001 to 100 USDC. Pricing strategy: flat, reputation-aware, or tiered.

2

Get back a proxy URL

Public, slug-based. Drop it into your Claude / Cursor / agent tool config in 10 seconds.

3

Agents call it. They pay or refuse.

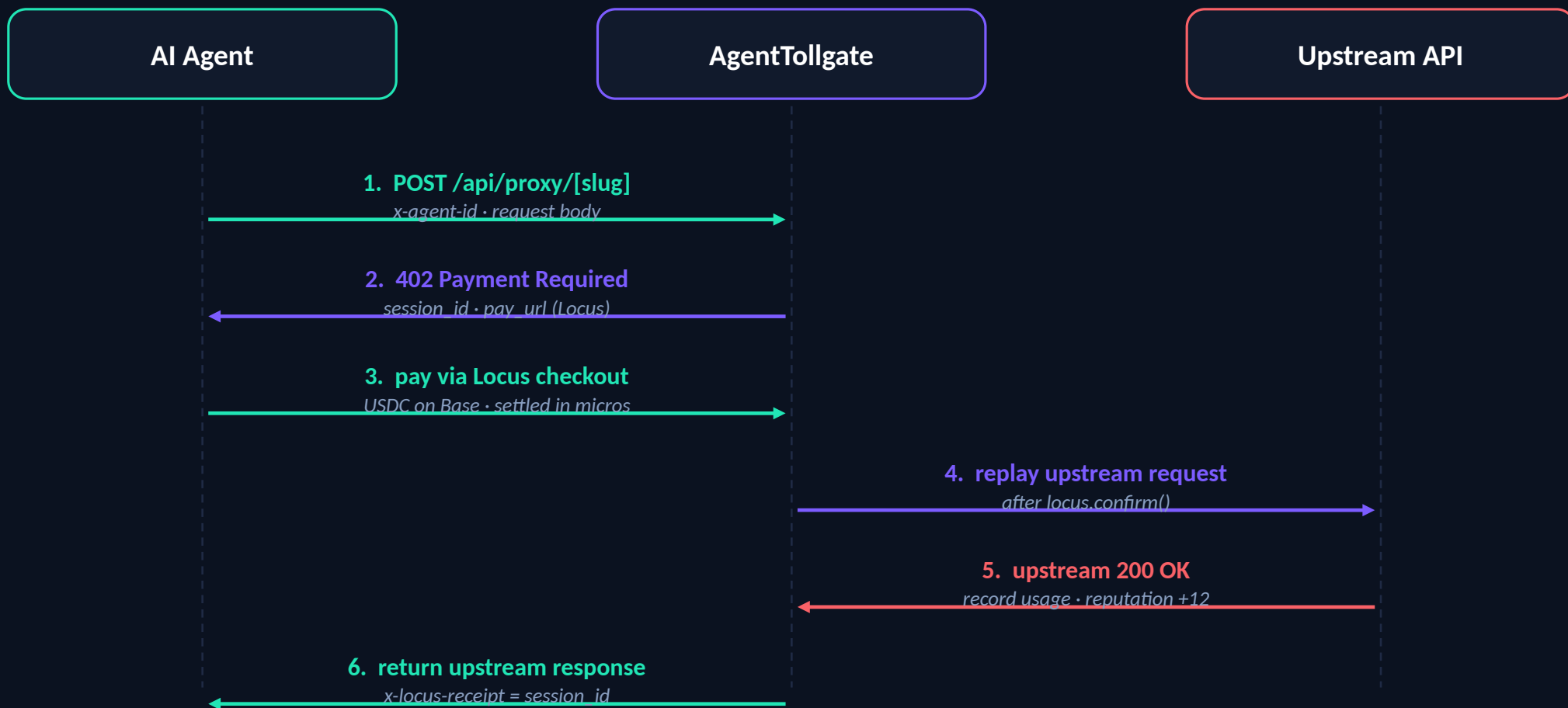
402 Payment Required → Locus session → USDC on Base → upstream replay. We handle the loop.

Result

Any developer ships a paywalled API in under 60 seconds. Any agent can pay it without merchant-side integration.

And every public tollgate auto-appears in our MCP catalog — so Claude, Cursor and ChatGPT can discover, pay, and consume natively.

The 402 → Pay → Replay loop.



Two extra HTTP calls — total. No browser, no redirects, no human required.

Three things merchants can't be bothered to build.



Policy-as-code DSL

Per-agent caps, daily limits, geo blocks, burst detection, reputation gates — all declared as JSON, evaluated server-side, returning typed denial codes the agent SDK self-corrects against.



Reputation-aware pricing

Every agent has a 0–1000 score from on-chain history. Score 1000 → 0.6× the base price; score 0 → 1.6×. Trusted agents get a discount. Sketchy ones surge.



MCP discovery, native

Every public tollgate auto-publishes to a built-in MCP server. Claude / Cursor / ChatGPT discover, search by category and price, and pay-and-call — without merchant-side integration.



Sub-cent settlement

Settled in micros via USDC on Base. 0.001 USDC works. Card networks can't do this profitably; we do it because the buyer is software.



402 + replay loop

x402-compatible flow. We capture the request body, mint the Locus session, and replay after payment. The agent sends the same request twice — that's the whole protocol.



Live analytics from call #1

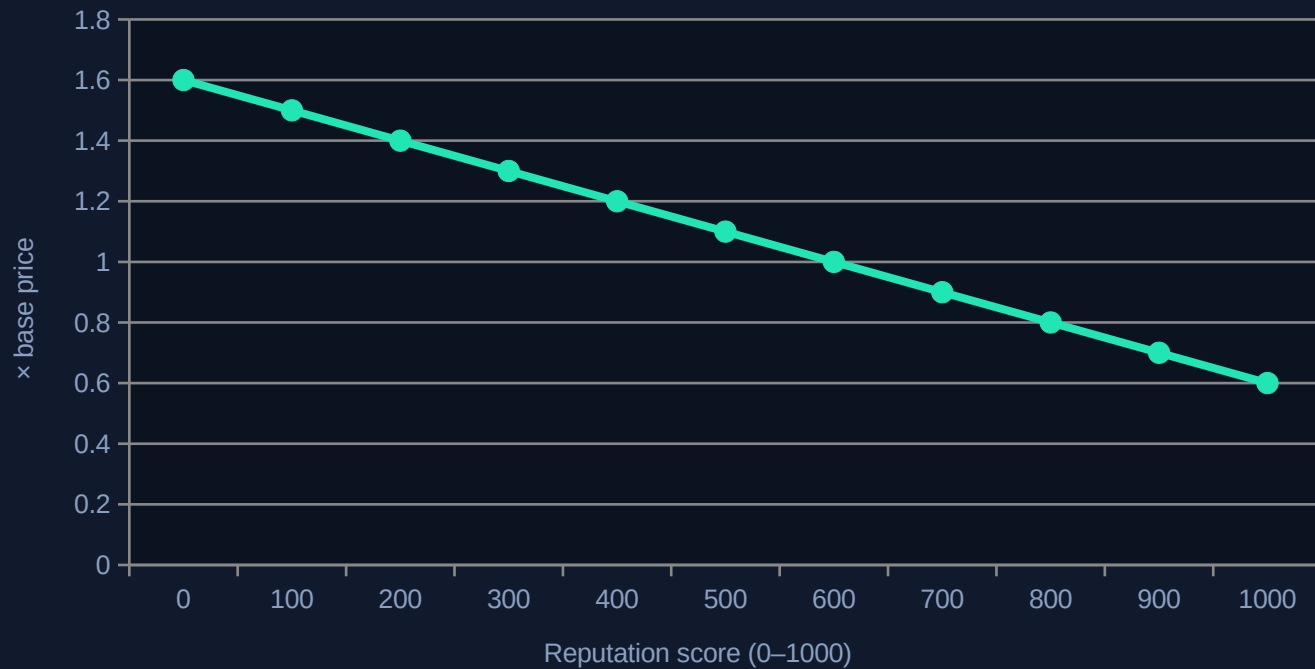
Revenue/min, top-spending agents, p95 latency, payment failure heatmap. The merchant sees exactly which agent paid, when, for what — and what they tried to abuse.

Where AgentTollgate wins.

Concern	Stripe + custom code	Crossmint / Coinbase	AgentTollgate
Designed for AI agents	✗	partial — web-only	yes — 402-pattern, headless first
USDC settlement	✗	yes	yes (Locus, USDC on Base)
Per-agent spend caps	DIY	✗	built-in DSL
Reputation pricing	✗	✗	on-chain audit trail → discounts
MCP discovery	✗	✗	first-class
Sub-cent price points	impractical (fees)	possible	trivial — settled in micros

Trusted agents pay less. Sketchy ones surge.

Price multiplier vs. reputation score



How the score moves

Successful call **+12 (cap +60/day)**

Heavy spender bonus **+0.001 per USDC**

Failed delivery (5xx) **-40**

Inactivity **decays toward 500**

Starting score **500**

Computed from the on-chain audit trail Locus already gives us — no additional data plane required.

Guardian rules in plain JSON.

policy.json

```
{
  "max_per_call_usdc": 5,
  "max_per_agent_per_day_usdc": 50,
  "max_per_tollgate_per_day_usdc": 500,
  "rate_limit_per_minute": 60,
  "require_min_reputation": 250,
  "block_unknown_agents": false,
  "block_geos": ["KP", "IR"],
  "block_burst": true,
  "emit_receipts": true
}
```

Typed denial codes

policy/over_per_call_cap

policy/over_daily_cap

policy/over_rate_limit

policy/insufficient_reputation

policy/unknown_agent_blocked

policy/burst_blocked

policy/geo_blocked

Returned as HTTP 429. Agents back off, climb reputation, or pick another tollgate — without human intervention.

One Next.js project. Five primitives.

DISCOVERY

AI Agent

Claude · Cursor · ChatGPT · custom

MCP Server (stdio JSON-RPC)

search_tollgates · call_tollgate

REQUEST LIFECYCLE

/api/proxy/[slug]

policy → pricing → 402 → replay

LOCUS BOUNDARY

src/lib/locus.ts

preflight · confirm · verifyWebhookSignature (real ⇌ mock)

SETTLEMENT

Locus Checkout — USDC on Base

settlement layer

State & observability

- merchants
- tollgates
- sessions (id = locus_id)
- usage (analytics)
- reputation
- rate_window

*better-sqlite3 → swap to Postgres for prod.
Every query is plain SQL.*

All Locus contact lives in one file. When the final API ships, only locus.ts changes.

Three rails crossed in 2025.

USDC on Base went mainstream

Fees dropped to fractions of a cent. Sub-cent API calls became settle-able for the first time. The unit economics for high-frequency agent traffic finally work.

Agents got wallets and budgets

Claude Agent SDK, OpenAI's Agent Builder, x402, Locus's CheckoutWithLocus — every agent runtime shipped a payment primitive in the last six months. The buyer is real.

MCP made discovery free

Once any tool can be exposed via MCP, every Claude / Cursor / ChatGPT user has a built-in marketplace client. Distribution became a feature flag.

BUILT ON CHECKOUTWITHLOCUS · OPEN-SOURCE · MIT

Try it tonight.

90 seconds from clone to first paid call. No Locus key required for the demo — mock mode is built in.

```
$ git clone <repo> && cd agenttollgate  
$ npm install && cp .env.example .env  
$ npm run seed && npm run dev  
→ http://localhost:3000
```

TRACK

Week 3 — CheckoutWithLocus

STACK

Next.js 15 · TS · USDC on Base

LICENSE

MIT